📊 MATRIZ DE CUMPLIMIENTO DE ENTREGABLES — EQUIPO 2

Proyecto Integrador Uniban • Backend & Modelos Predictivos / Deterministas • API FastAPI en Render

ENTREGABLES TOTALES
10 de 10 Requerimientos

CUMPLIMIENTO
100%

PRUEBAS AUTOMATIZADAS
191 tests (100% OK)

ARQUITECTURA & DESPLIEGUE
FastAPI en Render
Reglas D-001 a D-006

📋 Matriz Detallada de Requerimientos y Evidencias Técnicas

#	Requerimiento del Profesor	Estado	Dónde se encuentra en el Repositorio	Descripción / Evidencia Técnica
1	**Modelo determinista completamente funcional y validado** Cálculo de necesidades de cajas, cartón, bolsas y demás insumos.	Cumplido (100%)	• **Código núcleo:** `app/core/insumos/estimacion.py` • **Proyección base:** `app/core/proyeccion/base.py` • **Parámetros:** `config/parametros.yaml` • **Pruebas unitarias:** `test_caso_03_estimacion_insumos.py`	Modela la cosecha semanal con la física del racimo (1 / 10.5 semanas). A partir del volumen proyectado, deriva cajas, cartón y bolsas con coeficientes técnicos configurables y calcula faltantes y excedentes. Validado con casos de prueba calculados a mano.
2	**Modelo predictivo implementado y evaluado** Métricas de error y comparación vs modelo base.	Cumplido (100%)	• **Código modelo:** `app/core/proyeccion/predictivo.py` • **Reporte técnico:** `docs/reporte-evaluacion.md` (Línea 9) • **Decisión D-003:** `DECISIONES.md` (Línea 66) • **Pruebas:** `test_caso_10_predictivo.py`	Implementado con `scikit-learn` y validación con partición temporal (`TimeSeriesSplit`). Compara Error MAE contra Línea Base Ingenua (18.4%), Modelo Determinista (7.2%) y Predictivo (8.1%). Aplica la regla **D-003** manteniendo el determinista en producción por ausencia de datos históricos de Uniban.
3	**Módulo funcional de apoyo a la programación de cosecha y despacho** Dentro del alcance comprometido.	Cumplido (100%)	• **Heurística:** `app/logistica/heuristica.py` • **Fechas límite:** `app/logistica/fechas.py` • **Restricciones:** `app/logistica/restricciones.py` • **Pruebas:** `test_caso_08_heuristica_programacion.py`	Algoritmo de 4 pasos que deduce la fecha de corte hacia atrás restando tiempos de empaque y acarreo desde la ventana de despacho fija, ordena parcelas por urgencia y asigna vehículos disponibles respetando capacidades máximas.
4	**Capacidad de procesar aforamiento,**	Cumplido (100%)	• **Esquemas Ingesta:** `app/schemas/ingesta.py`	Modelos Pydantic v2 con validación estricta que procesan `estimatedPlants` ,

#	Requerimiento del Profesor	Estado	Dónde se encuentra en el Repositorio	Descripción / Evidencia Técnica
	cosecha, inventarios, consumos y ventanas Ingesta y validación de esquemas de datos.		• **Esquemas Recomendación:** `app/schemas/recomendacion.py` • **Adaptador Equipo 3:** `app/api/recomendacion.py` (Líneas 46-125)	`confidence`, `areaHa`, inventario (`inventory`), consumos, mermas/pérdidas y ventanas temporales con fecha/hora de inicio y fin.
5	**Generación automática de recomendaciones de abastecimiento** Cantidades requeridas, déficits o excedentes y fechas previstas.	Cumplido (100%)	• **Servicio central:** `app/core/servicio.py` (Línea 57) • **Pruebas unitarias:** `test_caso_04_servicio.py` • **Pruebas de excedentes:** `test_caso_07_excedente_insumos.py`	Función `recomendar_insumos()` que orquesta todo el flujo en memoria: calcula fecha límite de corte, racimos esperados y requerimiento exacto de insumos, determinando si hay déficit (`deficit > 0`) o excedente en el inventario actual.
6	**API funcional y documentada para recibir información del Equipo 3** Recepción de entradas y retorno de recomendaciones.	Cumplido (100%)	• **Swagger UI (En vivo):** `platano-logistica-api.onrender.com/docs` • **ReDoc:** `platano-logistica-api.onrender.com/redoc` • **Endpoint directo:** `app/api/recomendacion.py` • **Endpoint ingesta:** `app/api/ingesta.py` • **Router principal:** `app/main.py`	API REST en FastAPI desplegada en la nube en Render. Expone `POST /api/v1/recomendar`, `POST /api/v1/ingesta/operacion` y `GET /api/v1/planes/{periodo}`. Genera documentación interactiva OpenAPI disponible públicamente.
7	**Evidencia de pruebas que permita establecer coherencia y trazabilidad** Resultados auditables y consistentes.	Cumplido (100%)	• **Suite completa (191 tests):** `tests/` • **Pruebas API integración:** `tests/test_api_recomendacion.py` • **Trazabilidad D-005:** `DECISIONES.md` (Línea 87)	191 pruebas automáticas pasando al 100% en `pytest`. Cada respuesta y plan incluye el campo obligatorio `version_metodo` (`determinista-v1`), permitiendo auditar la regla y el origen de cada recomendación.
8	**Demostración obligatoria: Productor y parcela con variación de variables** Modificación de variables de entrada y recálculo automático.	Cumplido (100%)	• **Script CLI interactivo:** `scripts/demo_en_vivo.py` • **Pestaña Simulador Móvil:** `frontend/App.js` (Líneas 366-425)	Script ejecutable en consola (`python3 scripts/demo_en_vivo.py`) y simulador interactivo en la app móvil. Ejecutan el caso de la parcela **P-018** y permiten variar en tiempo real la ventana de despacho o el stock de cajas, mostrando el recálculo inmediato.

#	Requerimiento del Profesor	Estado	Dónde se encuentra en el Repositorio	Descripción / Evidencia Técnica
9	**Recepción y visualización mediante el componente del Equipo 3** Integración de extremo a extremo.	Cumplido (100%)	• **Endpoint en la nube:** `POST /api/v1/recomendar` • **Adaptador y normalizador:** `app/api/recomendacion.py` (Línea 46) • **Contrato acordado:** `docs/contrato-integracion.md` (Línea 78)	El endpoint acepta el JSON exacto de la app móvil del Equipo 3 (IDs en UUID, ciclo activo y lista de inventario), y devuelve el objeto JSON con la hora límite de corte y el déficit de cajas/bolsas para que el Equipo 3 lo pinte en su pantalla.
10	**Identificación y diferenciación explícita de datos simulados** Transparencia técnica sin falsas validaciones reales.	Cumplido (100%)	• **Registro de Decisiones:** `DECISIONES.md` (Línea 97) • **Reporte técnico:** `docs/reporte-evaluacion.md` (Línea 26) • **Fixtures y pruebas:** `test_caso_02_proyeccion_determinista.py` (Línea 18)	Cumplimiento formal de la regla **D-006**: como Unibán no ha entregado registros históricos reales, todos los datos simulados llevan el prefijo `sintetico_` y se declaran explícitamente en el código y reportes, sin presentarlos falsamente como validación real de campo.

🔗 Enlaces Directos y Repositorio de Evidencias (GitHub & Cloud)

#	Recurso / Entregable	Categoría / Tipo	Acceso Directo (Hipervínculo Web)
1	**Swagger API en Producción**	API Docs (OpenAPI / Cloud)	🌐 `https://platano-logistica-api.onrender.com/docs`
2	**Endpoint de Integración POST /recomendar**	API REST Endpoint	🌐 `https://platano-logistica-api.onrender.com/api/v1/recomendar`
3	**Contrato de Integración con Equipo 3**	Especificación Técnica	📄 `docs/contrato-integracion.md`
4	**Registro de Decisiones (D-001 a D-006)**	Decisiones de Arquitectura (ADR)	📄 `DECISIONES.md`
5	**Reporte Técnico de Evaluación de Modelos**	Métricas & Error MAE	📄 `docs/reporte-evaluacion.md`
6	**Script de Demostración en Vivo**	Script CLI (Python)	🐍 `scripts/demo_en_vivo.py`
7	**Lógica de Insumos y Fechas de Cosecha**	Servicio Central (Core)	⚙️ `app/core/servicio.py`
8	**Repositorio Completo GitHub (develop-jose)**	Control de Versiones	🐙 `GitHub: Uniban-Int1/CodeBack`